Beta CleanROMs

Complete User Manual — Version 2.6

■ *This application is in BETA. It may still contain bugs or unexpected behavior.*

CleanROMs is a command-line (PowerShell) tool for detecting duplicate ROMs inside a RetroBat installation, automatically keeping the best copy of each game, moving the rest to a backup folder (it never deletes them), and also cleaning up orphaned images, videos, and manuals left behind by ROMs that were already removed.

Development of this program started with ChatGPT (GPT-5.5) and was completed and debugged with Claude, Anthropic's AI assistant, through several rounds of real-world testing on large collections (several thousand ROMs per system).

Repository: https://github.com/batserra/CleanRoms

Table of Contents

1. What this program does

CleanROMs scans the ROM folders of your RetroBat systems and groups together the copies that correspond to the same game (for example, the USA version, the European version, and a compressed copy of the same title). For each group, it scores every copy according to a set of criteria (region, language, dump quality, version...) and decides which one is best. The rest are moved — never deleted — to a backup folder.

In summary, on every run:

1 It scans the chosen folder (or folders), looking for files with a recognized ROM extension.

2 It parses each file name (and, if it's a .zip/.7z containing a single ROM, the inner file name too) to extract region, language, version, revision, and special flags.

3 It normalizes each ROM's title (see section 6) and groups the ones that correspond to the same game, ignoring those tags — and applying manual aliases for cases where the name genuinely changes from one language to another (see section 7).

4 It scores every copy within its group and picks the one with the highest score.

5 It shows you the full plan (what's kept, what's moved, and why) before touching anything.

6 Only if you confirm with "Y", it moves the non-chosen copies to _duplicates\ — along with any associated file they had (save file, controller configuration...), see section 8.

Besides cleaning up ROMs, it can also detect orphaned images, videos, and manuals (from ScreenScraper/RetroBat) that no longer correspond to any existing ROM, undo the last cleanup you ran, and do everything at once for your entire collection. All of this is explained further below.

2. How it decides which ROM to keep: the scoring system

Each copy of a game receives a numeric score, adding up the points for every criterion it meets. The copy with the highest score in the group is the one that's kept ("KEEP"); the rest are moved ("MOVE"). These are the exact point tables the program uses (they live in Config\DecisionWeights.ps1, and can be freely adjusted if you want to change the priorities):

Region

Region	Points
Spain (ESP)	1000
Europe (EUR)	700
USA	400
Japan (JPN)	200
World	100
Unknown	0

By far the highest-weighted criterion — so a Spanish copy will almost always beat any other combination, unless it's also a bad dump, a hack, etc. (see section 5).

Language

Language	Points
Spanish (single language)	500
Multi-Spanish (includes Spanish among several)	350
Multi (several languages, no Spanish)	200
English	100
Japanese / Unknown	0

Dump quality

Criterion	Points
Verified [!] (confirmed good dump)	+200
Bad dump [b]	-500

ROM status

Status	Points
Beta	-150
Prototype	-300
Demo	-300
Sample	-300
Preview	-300
Kiosk	-300

These are all penalties: a Beta or Prototype version almost never wins over the final release, even if it has a better region or language.

Hacks, Homebrew, and Pirate

Type	Points
Hack	-400
Homebrew	-400
Pirate	-500

Note: this penalty would only apply if one of these ROMs ever got compared against another — but as explained in section 5, Hacks, Homebrew, Pirate, Betas, Prototypes, Demos, Samples, Previews, and Kiosks are excluded from comparison entirely, so in practice this score is almost never actually used. It's documented here in case you ever decide to enable comparison for them.

Version and revision

Version	Points
V1.0	100
V1.1	110
V1.2	120
V1.3	130
No version given	0

Revision	Points
Rev 0	0
Rev 1	10
Rev 2	20
Rev 3	30
Rev 4	40
No revision given	0

These are the two lowest-weighted criteria: they mainly serve to decide between two copies that already share region, language, and status — the higher the version or revision number, the more points (on the assumption that it fixes bugs from earlier ones). As of version 2.6 this score is actually computed from the file name — see section 21.

3. When there's a tie: tie-break order

If two copies end up with the same total score, they're compared in this exact order until something tells them apart:

1. Total score (explained in section 2).
2. Is one verified [!] and the other not? The verified one wins.
3. Is one a bad dump [b] and the other not? The bad dump loses.
4. Is one a Hack and the other not? The Hack loses.
5. Is one a Prototype and the other not? The prototype loses.
6. Is one a Beta and the other not? The beta loses.
7. Is one a Demo and the other not? The demo loses.
8. Is one compressed (.zip/.7z) and the other not? The one that is NOT compressed wins.
9. Do the titles have different lengths? The shorter title wins.
10. Are the titles alphabetically different? The one that comes first alphabetically wins.
11. If after all of this they're still exactly tied (same score, same criteria, same name): the program asks you which one you'd rather keep, showing you both full paths.

This last step (asking you) only happens when there truly is no automatic criterion left to tell them apart — in a normal collection this is uncommon, because step 8 (preferring the uncompressed one) already resolves the most typical case (the same ROM as a .zip and uncompressed).

4. Practical step-by-step example

Imagine you have these three copies of the same game in your SNES folder:

- A) Game (Europe) (En,Fr,De,Es).sfc
- B) Game (U) (V1.1) [!].sfc
- C) Game [E].zip

The program would group them like this (all three are "the same game" once the name is normalized), and score them:

Criterion	A) Europe/Multi-Es	B) USA V1.1 Verified	C) [E] compressed
Region	EUR = 700	USA = 400	EUR = 700
Language	Multi-Spanish = 350	Unknown = 0	Unknown = 0
Verified	No = 0	Yes = +200	No = 0
Version	— = 0	V1.1 = +110	— = 0
TOTAL	**1050**	**710**	**700**

Result: copy A is kept (1050 points, winning mainly because it includes Spanish among its languages), and B and C are moved to _duplicates\snes — along with any save file or configuration they had associated (see section 8). The reason you'd see on screen for A would literally be: "Region : EUR", "Language : Multi-Spanish", "Total score : 1050".

5. What's always excluded (Hacks, Betas, Demos, Homebrew...)

There are nine categories of ROMs that the program never compares, never moves, and never renames, no matter what their score is:

Category	How it's detected
Hack	The word "Hack" in the name, the No-Intro-style [h1] tag, or being inside a folder like "# Hacks #"
Fan translation	"Traducción"/"Translation"/"Trans", [T+Spa] tags or T(ESP)/T(POR), or a "Traducciones" folder
Beta	The word "Beta" in the name
Prototype	"Proto" or "Prototype" in the name
Demo	The word "Demo" in the name
Homebrew	The word "Homebrew" in the name
Pirate	The word "Pirate" in the name
Sample	The word "Sample" in the name
Preview / Kiosk	"Preview" or "Kiosk" in the name

The reason for excluding Beta/Prototype/Demo/Homebrew/Pirate/Sample/Preview/Kiosk, on top of Hacks and Translations, is that they represent content that's genuinely different from the final/retail release — a beta or a

prototype isn't "the same ROM with a worse name", it's a different version a collector may want to keep separately. Merging it with the final release would risk losing something unique if the decision engine chose to move it thinking it was just a duplicate.

One important nuance: if you have two copies of the same Beta, the same Hack, etc. (for example, a .smc and a .zip of the same hack), they won't be detected as duplicates of each other either — they're kept completely outside the cleanup system, not just protected from the original game. If you know for certain that two such files are identical copies, you can delete them by hand; the program will never do it for you.

6. How it recognizes that two ROMs are the same game

Before comparing anything, the program turns every file name into a "normalized title": a clean, lowercase version with nothing left except the game's actual name. Two ROMs are considered the same game if their normalized title is identical. These are, in order, the cleanup steps it applies:

1 Leading catalog number: strips a prefix like "0263 - " at the start of the name, typical of some numbered sets.

2 Date/time suffix: strips marks like "_20260709_034928" that some copying tools add when renaming duplicates.

3 Underscore-style names: some sets (mostly SNES) replace spaces, apostrophes, and parentheses with "_". The program recognizes this pattern: it converts the encoded apostrophe ("Kirby_s_Fun_Pak" → "Kirbys Fun Pak"), strips the trailing tags (region, version, hack, translation credit) joined by underscores, and only then converts the remaining underscores into spaces — so words like "and"/"the" are detected just as well as in a normal name.

4 Parentheses and brackets: ALL of their content is removed, whatever it is (year, region, language, publisher, dump group, revision...). In the vast majority of ROM sets (No-Intro, TOSEC, GoodTools...) everything between () or [] after the title is metadata, never part of the game's name.

5 Loose region/language words: in case they appear without parentheses (rare, but kept as a safety net): spanish, english, french, german, italian, portuguese, dutch, multi, usa, europe, japan, world, etc.

6 Revision and version: a loose "Rev A", "V1.1" (outside parentheses) is also stripped, always requiring a clear separator so it doesn't get confused with real words that start the same way (for example, this keeps "Revolver" or "Twin Turbo V8" intact).

7 Roman numerals: II, III, IV, V, VI, VII, VIII are converted to digits (2, 3, 4, 5, 6, 7, 8) so that "Final Fantasy IV" and "Final Fantasy 4" are recognized as the same title.

8 Articles: "the"/"a"/"an" in English, and "el"/"la"/"los"/"las" in Spanish, are stripped (so "The Lord of the Rings" and "Lord of the Rings, The" match, just like "Los Sims 2" and "Sims 2, The").

9 Apostrophes, periods, and exclamation/question marks: "Hoodlum's" = "Hoodlums", "Jr." = "Jr", "Day of Discovery!" = "Day of Discovery".

10 "&", "+", "and", "y": all treated as equivalent, so "Tom & Jerry" = "Tom and Jerry", and "Rayman Advance & Rayman 3" = "...+ Rayman 3".

11 Remaining symbols (dashes, colons, commas) are turned into spaces, and repeated spaces are collapsed into one.

All of these rules live in Modules\TitleNormalizer.ps1, in the Normalize-Title function, in case you ever want to adjust or extend any of them.

7. Title aliases: when the name is genuinely different

The rules in the previous section solve every case where the difference between two names is just a region, language, version, or format tag. But there are cases where the game's name genuinely changes from one region to another — especially with official Spanish translations, or with abbreviated titles — and no automatic rule can guess that it's the same game without risking merging unrelated games by mistake. Some real examples:

Name A	Name B	Why don't they match on their own?
Narnia - El león, la bruja y el armario	The Chronicles of Narnia: The Lion, the Witch and the Wardrobe	Fully translated title
Harry Potter 5 - La Orden del Fénix	Harry Potter and the Order of the Phoenix	Spanish numbering vs. full English name
Golden Sun 2	Golden Sun - La Edad Perdida	Added numbering vs. official subtitle
Zelda - The Minish Cap	The Legend of Zelda, The - The Minish Cap	Abbreviated name vs. full title
Piratas del Caribe - El cofre del hombre muerto	Pirates of the Caribbean - Dead Man's Chest	Translated title

For these cases there's Config\TitleAliases.json: an editable dictionary that maps an already-normalized variant to its canonical form. The program checks it as the last step, right after applying all the rules from section 6 — if the result matches a key in the file, it's replaced with the associated value before grouping.

File format:

```
{
  "narnia leon bruja armario": "chronicles of narnia lion witch wardrobe",
  "golden sun edad perdida": "golden sun 2 edad perdida",
  "piratas del caribe cofre del hombre muerto": "pirates of caribbean dead mans chest"
}
```

How to add a new alias yourself:

1 Note down the two (or more) exact file names that should be considered the same game and aren't.

2 Pick which of the two you want to use as the "canonical form" (the alias value) — it doesn't matter which, it just has to be the same for every variant of that game.

3 Open Config\TitleAliases.json with a text editor (Notepad, Notepad++...).

4 Add a new line with the alternate name already in lowercase and without region/version tags (imagine it has already gone through the rules from section 6), pointing to the canonical name you chose.

5 Save the file. No special restart is needed, the program reads it on every run.

If you're not sure how a name will come out after normalizing, that's fine: write the key as close as possible to what you think it'll be (all lowercase, no parentheses/brackets, no accents) and try it — if it doesn't match the first time, run the program and compare it with the normalized name that already worked for the other file in the same group.

Important: if you ever update the program and the normalization rules change (for example, a new word gets added to the strip list), aliases you already had saved may stop matching the exact key the new rules generate. If you notice an alias that used to work stops working after an update, just check how the name normalizes now and update the key.

8. Associated files: save files and configurations

Many ROMs have their own files in the same folder, besides the ROM file itself: save files (.sav, .srm, .sram, .state) or per-game controller configuration profiles (for example, Game.dsk.p2k.cfg for Amstrad CPC). When a ROM gets moved to _duplicates for being a duplicate, the program automatically moves these associated files along with it, in the same step — so they never end up orphaned, pointing to a ROM that's no longer in its original folder.

The program recognizes a file as associated with a ROM in two ways:

- Simple match: same file name without the extension ("Game.sav" belongs to "Game.gba").
- Compound extension: the associated file starts with the FULL ROM name, extension included, followed by more extensions of its own ("Barbarian (Europe).dsk.p2k.cfg" belongs to "Barbarian (Europe).dsk").

This feature is controlled by the MoveAssets setting in Config\Settings.ps1 (on by default). One important caveat: the program never treats another real ROM as if it were an associated file, even if it shares the same base name with a different extension — for example, "Game.sfc" and "Game.smc" are two independent copies, each with its own entry in the cleanup plan, not an "associate" of the other.

9. Cleaning up orphaned images, videos, and manuals

RetroBat keeps each game's images, videos, and manuals (usually downloaded with ScreenScraper) in the images\, videos\, and manuals\ folders of each system. When you delete a ROM (or CleanROMs moves a duplicate), its image/video/manual is left behind unused — an "orphan" file. This option detects them and moves them (never deletes them) to _duplicates\<system>\images, \videos, or \manuals.

How it knows which file belongs to which ROM

ScreenScraper (through RetroBat) adds a suffix to the ROM name depending on the media type, for example: Game-marquee.png, Game-video.mp4, Game-manual.pdf. The program recognizes these suffixes and strips them before comparing, so "2 Game Pack! - Uno & Skip-Bo (Europe)-marquee.png" is correctly recognized as the image for "2 Game Pack! - Uno & Skip-Bo (Europe).zip" — dashes that are already part of the title itself are left untouched; only the suffix is trimmed if it matches exactly at the end.

Currently recognized suffixes: -image, -video, -marquee, -thumb, -wheel, -manual, -boxfront, -boxback, -box2dfront, -box2dback, -box3d, -fanart, -title, -screenshot, -screenshottitle, -cartridge, -support, -mix, -bezel, -map.

> *The last two (-bezel, the decorative frame around the game, and -map, maps for some games) were added after real testing showed RetroBat generates them routinely and they weren't in the original list — without them, those files were treated as orphans even though their ROM still existed. If you come across a new suffix ScreenScraper uses that isn't in this list, it can be added in a moment — they're all together in $Global:MediaFileSuffixes, at the top of Modules\MediaCleaner.ps1.*

> *This cleanup also checks each system's manuals\ folder (it used to only check images\ and videos\). It does not check downloaded_images\, _duplicates\, or folders for unsupported systems (see section 17).*

10. Requirements and installation

You need PowerShell 7.3 or later (Windows PowerShell 5.1, which ships with Windows by default, is NOT enough): https://github.com/PowerShell/PowerShell/releases

Download the installer for your system (for example, PowerShell-7.x.x-win-x64.msi) and run it with the default options.

Optional: 7-Zip

If you have 7-Zip installed, the program can look inside your .7z files to detect region/language even when those tags are only present on the inner file name, not on the .7z itself. It's found automatically in the PATH, in typical install locations ("C:\Program Files\7-Zip"), and in the Windows registry — no manual setup needed. Without 7-Zip, .7z files are still processed normally, just by their own file name.

Installation

Copy the entire "CleanRoms" folder inside the "roms" folder of your RetroBat installation. Then double-click **"Run CleanROMs.bat"** — this is the recommended way to start it, especially the first time.

That launcher automatically unblocks every file in the folder and starts main.ps1 for you. This avoids a common error the first time you run a script downloaded from the internet: Windows marks files from a downloaded/unzipped folder as "blocked", and PowerShell's default policy then refuses to run main.ps1 directly, showing something like:

> *main.ps1 : File ...main.ps1 is not digitally signed. You cannot run this script on the current system.*

If you'd rather run main.ps1 directly (right-click it and choose "Run with PowerShell 7", or from a terminal with .\main.ps1) and you hit that error, just unblock the folder once and it won't happen again:

> *Get-ChildItem -Path "C:\RetroBat\roms\CleanRoms" -Recurse | Unblock-File*

> *If you already had a previous version installed, delete the whole CleanRoms folder before extracting the new one — some unzip tools don't overwrite existing files by default (in particular Config\TitleAliases.json), and you could end up with a mix of versions that doesn't include the latest aliases or settings.*

11. First run: configuring the RetroBat path

The first time you run the program (or if you delete Config\UserSettings.json) it will ask you where RetroBat's "roms" folder is, already suggesting a default value (the folder that contains CleanRoms):

> *RetroBat path not configured yet.*
> *Path to RetroBat's 'roms' folder [C:\RetroBat\roms]:*

Press Enter to accept the suggested path, or type another one. It's saved so you won't be asked again. If you move RetroBat elsewhere, delete Config\UserSettings.json and it will ask you again.

12. Configuring the program in depth

Almost all of the program's behavior is controlled from files inside the Config\ folder, editable with any text editor:

Config\UserSettings.json

Just stores the RetroBat path you set on first run (section 11) and the language you chose (section 19). Delete it if you need it to ask you again.

Config\Settings.ps1

Almost everything else lives here. The settings most relevant day-to-day:

Setting	What it controls
PreviewOnly	If $true, never actually moves/deletes anything even if you confirm with "Y" (see section 14)
MoveAssets	If $true (default), moves associated files along with each ROM (section 8)
RemoveDuplicates	If ever enabled, would allow deleting instead of moving — off by default: the program never deletes
$Global:RomExtensions	List of 110+ file extensions recognized as ROMs
$Global:IgnoredFolders	Folders that are never scanned for ROMs (images, videos, manuals, _duplicates, # Hacks #...)
$Global:SystemPaths	List of 60+ supported systems and their folder name
$Global:OrphanedMediaFolders	Which subfolders the orphan cleanup checks (images, videos, manuals)
$Global:MediaFileSuffixes	Recognized image/video/manual suffixes (section 9)

Config\DecisionWeights.ps1

Contains the full scoring tables from section 2 (region, language, dump quality, status, version, revision). Feel free to edit them if, for example, you want to prioritize English over Spanish, or penalize prototypes less.

Config\TitleAliases.json

The title aliases from section 7 — for games whose name genuinely changes between languages or editions.

13. The main menu (the 6 options)

On startup, the program asks you what you want to do:

What do you want to do?
1) Clean up duplicate ROMs
2) Undo the last cleanup
3) Clean up orphaned images/videos/manuals
4) ALL: Move ROMs and images/videos/manuals for every system
5) Configuration (RetroBat path / language)
6) Exit

Option 1 — Clean up duplicate ROMs

It asks you for the system (or "0) ALL SYSTEMS"). The system menu only shows systems that actually have ROMs in their folder — if a system has no valid file at all, it doesn't even appear in the list. It then scans, groups, scores, shows you the full plan, exports the reports (section 15), and only moves files if you answer "Y" to the confirmation. It also organizes any loose hacks it finds (see section 5).

Option 2 — Undo the last cleanup

Puts back in its original place any file that was actually moved — checking on disk, not just on paper, so if that time you only previewed or cancelled, it will simply tell you there's nothing to undo. It can undo more than just the most recent session — see the full detail in section 16.

Option 3 — Clean up orphaned images/videos/manuals

See section 9 for the full detail.

Option 4 — ALL: clean up everything, every system

Chains option 1 and option 3, but directly across every configured system, without asking you which one again. Each step (ROMs, then orphans) has its own preview and its own separate Y/N confirmation.

Option 5 — Configuration

Shows the currently configured RetroBat path and language, and lets you change them without having to delete Config\UserSettings.json by hand (see sections 11 and 19): change just the path, just the language, or both. It then goes straight back to the main menu.

Option 6 — Exit

Closes the program.

14. Simulation mode (PreviewOnly)

In Config\Settings.ps1 you can turn on:

```
PreviewOnly = $true
```

With this, even if you confirm with "Y", the program only prints what it would do ([PREVIEW MOVE]) without actually moving anything. Highly recommended the first time you try it on a new collection, or after changing an important setting.

15. Generated reports and the full log

Every time a cleanup plan is generated (even if it finds no duplicates), three files are exported to Resultado\:

- CleanPlan.json — the full plan detail (also used by "Undo the last cleanup").
- CleanPlan.csv — summarized version, to open in Excel.
- CleanPlan.html — visual report with stats and a color-coded table, to open by double-clicking in your browser.

In addition, every full session (everything you see on screen, including your answers to the menu and Y/N confirmations) is saved to a text file inside Logs\CleanROMs_<date>.log — useful for reviewing exactly what happened after closing the console, or for attaching it if you report an issue (see section 19).

16. Undoing the last cleanup

Expanding on menu option 2: for every file the plan moved, the program checks whether it's really in its destination folder before restoring it, and if the original spot is already occupied by something else, it warns you and skips it instead of overwriting. At the end it gives you a summary: how many were restored, how many didn't need touching, and how many were skipped due to a conflict.

This option also restores associated files (save files, controller configs...) that were moved alongside a ROM, not just the ROM itself — and it covers moves made while organizing loose hacks into "# Hacks y Otros #" and deduplicating them by hash (see section 5) too, not just the regular ROM cleanup.

More than one session to undo

Every time you start a new cleanup, the previous session is automatically archived to Resultado\History\ before it begins. If there's any archived session, option 2 lets you pick which one to undo:

Which cleanup do you want to undo?
0) The most recent one (Enter)
1) 2026-08-20 15:32
2) 2026-08-19 10:05
Pick a number:

Pressing Enter without typing anything undoes the most recent one, exactly as before — if you never care about this, nothing changes in how you use the program. By default the last 10 sessions are kept (adjustable with UndoHistoryLimit in Config\Settings.ps1); older ones are dropped automatically.

17. Supported systems and extensions

The program currently recognizes 60+ systems (Nintendo, Sega, Sony, arcade, home computers...) and 110+ different file extensions. The full list is in Config\Settings.ps1 ($Global:SystemPaths and $Global:RomExtensions) and can be freely extended. Some examples:

Category	Examples
Nintendo	NES, SNES, N64, GameCube, Wii, Game Boy (Color/Advance), NDS
Sega	Master System, Mega Drive/Genesis, Game Gear, Saturn, Dreamcast
Sony	PlayStation, PlayStation 2, PSP, PS Vita (.vpk)
Arcade	MAME, FinalBurn Neo, Neo Geo, CPS1, CPS2, CPS3
Home computers	Amstrad CPC, ZX Spectrum, MSX, Commodore 64, X68000

Some systems in the list have limited support because their games come as a "folder" rather than a single file (PS3, PS Vita without .vpk, PC DOS, Nintendo Switch in some cases) — the program detects files by extension, not whole folders, so those formats won't be recognized even though the system is in the list.

Multi-core arcade systems (MAME, FinalBurn Neo, CPS1/2/3, Neo Geo, Cave, Zinc, Namco System 11/22...) are each processed independently, without comparing ROMs across different systems — this is intentional: RetroBat sometimes needs a copy of the same arcade file in several different system folders so that each core can launch it, so they are never treated as "duplicates" of each other even though they may look like it.

18. Special file names (brackets, apostrophes, underscores)

ROM names frequently use characters that, in PowerShell, have a special meaning if not handled carefully — especially brackets ([!], [E], [T+Eng]...), which PowerShell can interpret as search wildcards instead of literal text. The program internally uses "literal" paths in every file operation (checking existence, moving, deleting, undoing...) precisely so this is never an issue: a ROM with brackets in its name is processed exactly the same as one without them.

This matters because, without this care, a ROM with brackets in its name could silently disappear from the scan (the program would think the file doesn't exist), which in turn would prevent detecting its real duplicates. If you ever see a "File does not exist" warning in the log for a ROM you know is really in its folder, please report it (section 24) along with the exact file name.

19. Language: Español / English

The program is available in Spanish and English. The first time you run it, it will ask which language you prefer:

Selecciona idioma / Select language:
1) Español
2) English

The answer is saved in Config\UserSettings.json (along with the RetroBat path) and won't be asked again. If you want to change it later, edit that file by hand ("Language": "es" or "Language": "en") or delete it so it asks you again.

The translation covers the program's whole usual flow: menus, preview, confirmations, final summary, orphan cleanup, undo, and the most important notices. Some very low-level text (certain internal debug messages that in practice never actually show up during normal use) may still appear only in Spanish if they were ever triggered.

20. About renaming ROMs (RENAME)

You may see "RENAME" mentioned in the cleanup summary, alongside KEEP, MOVE, and DELETE. To be clear: the program never renames any ROM. That counter always shows 0.

Title normalization (section 6) — stripping dashes, parentheses, region tags... — is used purely internally, to decide which copies are the same game. It's never applied to the actual file name: the ROM that's kept stays with exactly the name it already had.

This is intentional: renaming the file RetroBat already has indexed would break its association with the images, videos, and manuals already downloaded via ScreenScraper, since gamelist.xml links them by the exact file name. Changing the name without also updating gamelist.xml would make you lose that already-obtained artwork for the kept game.

21. What's new in version 2.6 ★ NEW IN 2.6

This section summarizes what changes compared to version 2.5, described throughout the rest of this manual. All the changes in this section come from a thorough code review and real-world testing carried out on 2.5.

ROM version and revision are now actually detected

Section 2 already documented a scoring table for a ROM's version (V1.0, V1.1...) and revision (Rev 0, Rev 1...). In 2.5 this table existed but had no real effect: the program never actually extracted a version or revision from the file name, so every ROM scored 0 points on both criteria regardless of what its name said. Version 2.6 adds the missing detection: tags like (V1.1), [v1.2], (Rev 1), or [Rev A] are now recognized directly from the file name and correctly feed into the scoring table from section 2. Titles that genuinely contain the word "Revenge", "Revolution", or "Review" are correctly left alone (they are not mistaken for a "Rev" tag).

"[BIOS]" is no longer mistaken for a bad dump

The bad-dump detection ([b], [b1]...) previously matched any bracketed tag starting with the letter "b", regardless of case — which meant ROMs correctly tagged [BIOS] or [Bonus] were incorrectly penalized -500 points as if they were bad dumps. Version 2.6 only recognizes the real GoodTools bad-dump codes ([b], [b1], [b2]...), leaving other tags alone.

Duplicate hacks now go to the right system folder

When the "# Hacks y Otros #" cleanup (see section 5) finds two byte-identical copies of the same hack inside that folder, it moves the extra copy to the duplicates folder. In 2.5, this could compute the wrong destination: instead of _duplicates\<system>\, it sometimes created a _duplicates\# Hacks y Otros #\ folder, because the destination was derived from the immediate parent folder of the file being moved. Version 2.6 always uses the ROM's real system name for this, so duplicate hacks now land in the same place as every other duplicate: _duplicates\<system>\.

Clearer confirmation when moving duplicate hacks

The confirmation prompt shown before moving exact duplicate hacks to _duplicates\ now explains that these are byte-identical files (so answering yes is safe and nothing is lost), and warns that this particular step cannot be undone with the "Undo the last cleanup" menu option.

Simulation mode (PreviewOnly) is now respected everywhere

PreviewOnly already kept the regular ROM cleanup from actually moving anything, but the orphaned images/videos/manuals cleanup (section 9) and both hack steps (organizing loose hacks and deduplicating by hash) never checked it at all — with PreviewOnly on, those parts still moved files for real as soon as you confirmed. All three now respect PreviewOnly, and the "Simulation mode is on" notice (which existed but was never actually printed anywhere) now shows up before each confirmation when it applies.

"Undo the last cleanup" expanded

It now also restores associated files (save files, controller configs) moved alongside a ROM, and covers hack organizing/deduplication moves too, not just the regular ROM cleanup. Also, every new cleanup archives the previous session to Resultado\History\, so menu option 2 can undo an older session, not just the most recent one. See the full detail in section 16.

Command-line, unattended runs

New parameters on main.ps1 (-Action, -System, -Yes, -PreviewOnly) let you schedule CleanROMs with Windows Task Scheduler without it ever waiting for an answer. See the full detail in section 22.

Parallel hashing for large collections

Hashing (used to find exact duplicate hacks, and to verify moves) now runs in parallel on large batches instead of one file at a time, using PowerShell 7's ForEach-Object -Parallel. Controlled by HashParallelThreshold and HashParallelism in Config\Settings.ps1; by default it's not worth parallelizing below 20 files, and it uses 4 at a time above that.

None of these changes affect how Hacks, Betas, Translations, etc. described in section 5 are detected or organized — that logic works the same as in 2.5. It also doesn't change the fact that the program never deletes files — it still always moves them to a backup folder.

22. Command-line use: automated/unattended runs

For unattended runs (for example, with Windows Task Scheduler), main.ps1 accepts parameters that skip the menu entirely. Without them, the program behaves exactly as it always has — none of this is required or changes normal use.

Parameter	What it does
-Action Clean|Orphans|All|Undo	What to do: Clean = clean up duplicate ROMs (and organize hacks), Orphans = orphaned media, All = everything, Undo = undo the last cleanup
-System <folder>	The system's folder name (e.g. "snes", "gba"); omit or use "ALL" for every system. Ignored with -Action Undo
-Yes	Auto-confirms every Y/N prompt instead of waiting for someone to answer — this is what actually makes a run unattended
-PreviewOnly	Forces simulation mode (section 14) for this run only, without having to edit Config\Settings.ps1

Examples:

```
pwsh -File main.ps1 -Action Clean -System snes -Yes
pwsh -File main.ps1 -Action All -Yes
pwsh -File main.ps1 -Action Undo -Yes
```

The -System parameter takes the folder name exactly as it appears under RetroBat\roms (for example "snes"), not the long descriptive name shown in the interactive menu — that's what anyone setting up a scheduled task will already have at hand.

> *Without -Action, main.ps1 behaves exactly as it always has (interactive menu). Without -Yes, -Action still skips the menu but still asks for confirmation at each step, same as interactive mode — -Yes is specifically what keeps the program from sitting there waiting for an answer nobody's going to give.*

23. Tests performed

Before considering any fix closed, it has been tested with real data, not just isolated cases. The process followed in each round of testing has been:

1 First, on small or medium system folders (GBA, SNES, Amstrad CPC), using the complete listing of real files in those folders (not a sample), to find and fix name patterns the rules didn't yet recognize.

2 Then, on the total ROM count of a full collection (35+ systems at once, over 13,600 ROMs in total), checking the resulting log for how many files are processed, how many duplicate groups are detected, and how many actually end up being moved, system by system.

3 Finally, reviewing the full log file for errors or warnings (there should be none), and comparing the file listing before and after cleanup to confirm that no image, video, or manual belonging to a kept ROM is moved by mistake, and that no ROM is moved without genuinely being a duplicate.

Result of the most recent test on a full collection (13,612 ROMs scanned across 35 different systems, without PreviewOnly enabled):

System	ROMs scanned	Moved to _duplicates	% moved
amstradcpc	3,441	515	15.0 %
gba	1,542	394	25.6 %
snes	2,100	498	23.7 %
nes	1,270	205	16.1 %
n64	168	21	12.5 %
megadrive	871	9	1.0 %
ngpc	93	14	15.1 %
nds	24	2	8.3 %
psx	78	2	2.6 %
vectrex	26	1	3.8 %
(remaining systems: no duplicates)	5,999	0	0.0 %
TOTAL	**13,612**	**1,661**	**12.2 %**

The percentage moved varies a lot from one system to another: systems like Amstrad CPC, GBA, or SNES have collections with many accumulated regional versions of the same games (hence the 15-26% of real duplicates), while smaller systems or ones with already-clean romsets (like Mega Drive in this test) barely have any duplicates to move. A high percentage isn't a bug — it reflects how many extra copies really were in that folder; what matters, verified by reviewing the log, is that every moved file belonged to a real duplicate group and no ROM was moved on its own.

During this testing, several issues were found and fixed — from the bug where brackets were treated as wildcards (section 18) to unrecognized image suffixes (section 9) — precisely by comparing the expected behavior against what actually showed up in the log and the file listing, not just reviewing the code on paper.

24. This is a BETA — how to report issues

This program is still being tested and may fail in unforeseen cases. If you find a bug, or want a feature added or removed, please email as much detail as you can (what you did, what you expected to happen, and what happened instead) to:

✉ *batserra@gmail.com*

Project repository: https://github.com/batserra/CleanRoms

If the issue involves ROMs not grouping as expected, or files disappearing from the folder without explanation, please attach if you can: the log file from that run (Logs\CleanROMs_<date>.log) and, if possible, a listing of the affected files (for example with dir /s /b > listing.txt from that system's folder). The more real data, the faster the exact cause can be found.